

Relazione Tecnica

QSVM 6 qubit sul dataset Bank Account Fraud (BAF)

Giuseppe Pasquale Caligiure, Ettore Aloe, Nicola Mercuri

2 giugno 2026

Indice

1	Scopo del progetto	3
2	Configurazione dell'ambiente	3
2.1	Creazione dell'ambiente virtuale (esempio WSL)	3
2.2	Installazione delle dipendenze	3
3	Struttura generale dello script	4
4	Blocco 1 – Caricamento del dataset	4
5	Blocco 2 – Preprocessing	5
6	Blocco 3 – Mutual Information e scelta della top feature	5
7	Blocco 4 – PCA a 5 componenti + feature supervisionata	5
8	Blocco 5 – Bilanciamento delle classi	6
9	Blocco 6 – Scaling angolare per la ZZFeatureMap	7
10	Blocco 7 – Train/Test split	7
11	Blocco 8 – Quantum kernel con Qiskit	7
11.1	ZZFeatureMap a 6 qubit	7
11.2	Backend quantistico	8
11.3	Compute–uncompute per il kernel	8
11.4	Matrici K_{train} e K_{test}	8
12	Blocco 9 – QSVM e baseline classiche	8
13	Blocco 10 – Grafici e visualizzazioni	9
13.1	Confronto delle metriche QSVM vs SVM classiche	9
13.2	Confusion matrices	9
14	Blocco 11 – Salvataggio risultati e file di output	10

15 Considerazioni metodologiche e interpretative	10
15.1 Motivazioni della scelta PCA(5) + feature MI	10
15.2 Aspetti critici del kernel quantistico	10
15.3 Interpretazione del confronto con le SVM classiche	10

1 Scopo del progetto

Questo progetto implementa e analizza una pipeline completa di **Quantum Support Vector Machine (QSVM)** su 6 qubit applicata al dataset **Bank Account Fraud (BAF) – NeurIPS 2022**, con l'obiettivo di confrontare in modo rigoroso un classificatore basato su kernel quantistico con classificatori classici (SVM lineare e SVM RBF) per il problema di rilevazione di frodi nell'apertura di conti bancari.

La pipeline segue la sequenza:

1. Caricamento e pulizia del dataset BAF.
2. Preprocessing classico (encoding, imputazione, standardizzazione).
3. Calcolo della **Mutual Information** (MI) per valutare l'importanza delle feature rispetto al target.
4. Riduzione dimensionale tramite **PCA a 5 componenti**.
5. Selezione della **migliore feature supervisionata** (top-MI) e costruzione di uno spazio finale a **6 dimensioni**.
6. Bilanciamento delle classi e riscalamento angolare in $[-\pi, \pi]$ per l'input della ZZFeatureMap a 6 qubit.
7. Costruzione del **kernel quantistico** tramite circuito compute-uncompute con Qiskit Aer.
8. Addestramento della **QSVM** (SVC con `kernel="precomputed"`) e confronto con SVM lineare e SVM RBF classiche.
9. Analisi quantitativa e grafica dei risultati, con salvataggio di metriche, matrici di confusione e heatmap della matrice kernel.

L'intero flusso è implementato nel file Python:

```
qsvm_baf_6qubit_pca5_mi1.py
```

2 Configurazione dell'ambiente

Per eseguire correttamente la pipeline è consigliabile un ambiente Linux (nativo o via **WSL** su Windows), con Python 3.11/3.12 e un ambiente virtuale dedicato.

2.1 Creazione dell'ambiente virtuale (esempio WSL)

```
1 mkdir -p ~/.venvs
2 python3.12 -m venv ~/.venvs/quantum_env
3 source ~/.venvs/quantum_env/bin/activate
```

2.2 Installazione delle dipendenze

All'interno dell'ambiente virtuale:

```
1 pip install "qiskit==1.1.1"
2 pip install qiskit-aer-gpu      # opzionale, se disponibile GPU
  NVIDIA
3 pip install scikit-learn pandas numpy matplotlib seaborn imbalanced
  -learn
```

Se la GPU non è disponibile o non è configurata, lo script effettuerà automaticamente il fallback su **CPU**.

Per eseguire la pipeline è sufficiente avere nella working directory:

- `Base.csv` – dataset BAF preprocessato di partenza;
- `qsvm_baf_6qubit_pca5_mi1-1-2.py` – script principale.

3 Struttura generale dello script

Lo script è suddiviso in blocchi numerati, con stampe esplicative a console. Di seguito una panoramica sintetica:

Blocco 1 Caricamento dataset BAF (`Base.csv`).

Blocco 2 Preprocessing (drop colonne, encoding categoriche, imputazione, standardizzazione).

Blocco 3 Calcolo della Mutual Information e scelta della top feature supervisionata.

Blocco 4 PCA a 5 componenti + aggiunta della top feature (spazio a 6 dimensioni).

Blocco 5 Bilanciamento delle classi (150 campioni per classe, 300 totali).

Blocco 6 Scaling angolare in $[-\pi, \pi]$ per la `ZZFeatureMap`.

Blocco 7 Train/Test split 75/25 stratificato.

Blocco 8 Costruzione del kernel quantistico (`ZZFeatureMap` 6 qubit + `compute-uncompute`).

Blocco 9 Addestramento della QSVM e delle SVM classiche, calcolo delle metriche.

Blocco 10 Generazione dei grafici (heatmap kernel, benchmark metriche, confusion matrix, decision boundaries 2D).

Blocco 11 Salvataggio dei risultati (CSV, NPY, immagini e `readme_6q.txt`).

Nei paragrafi successivi ogni blocco viene descritto in dettaglio.

4 Blocco 1 – Caricamento del dataset

Questo blocco è puramente descrittivo e serve a verificare che i dati siano stati caricati correttamente e che lo sbilanciamento fra classi sia in linea con le aspettative (circa 1–2% di frodi nella versione completa, prima del successivo bilanciamento su un sotto-campione).

5 Blocco 2 – Preprocessing

Il preprocessing prepara i dati per le fasi successive di feature selection e proiezione. Le operazioni includono:

- **Rimozione di colonne non utilizzabili.**
- **Encoding delle variabili categoriche:** tutte le colonne di tipo `object` o `category` (eccetto `fraud_bool`) vengono trasformate con `LabelEncoder`, ottenendo feature numeriche intere.
- **Conversione numerica forzata:** garantisce che *tutte* le feature in `X` siano numeriche e compatibili con PCA e MI.
- **Imputazione dei valori mancanti:** si calcola la mediana *escludendo* $i - 1$; $i - 1$ vengono quindi sostituiti con la mediana. Questa scelta evita che il valore -1 sposti la stima del valore tipico verso il basso e rappresenta una strategia robusta per l'imputazione di dati mancanti.
- **Separazione feature/target e standardizzazione.**

6 Blocco 3 – Mutual Information e scelta della top feature

In questo blocco si calcola la **Mutual Information** fra ogni feature e il target `fraud_bool` (usando `mutual_info_classif` di `scikit-learn`).

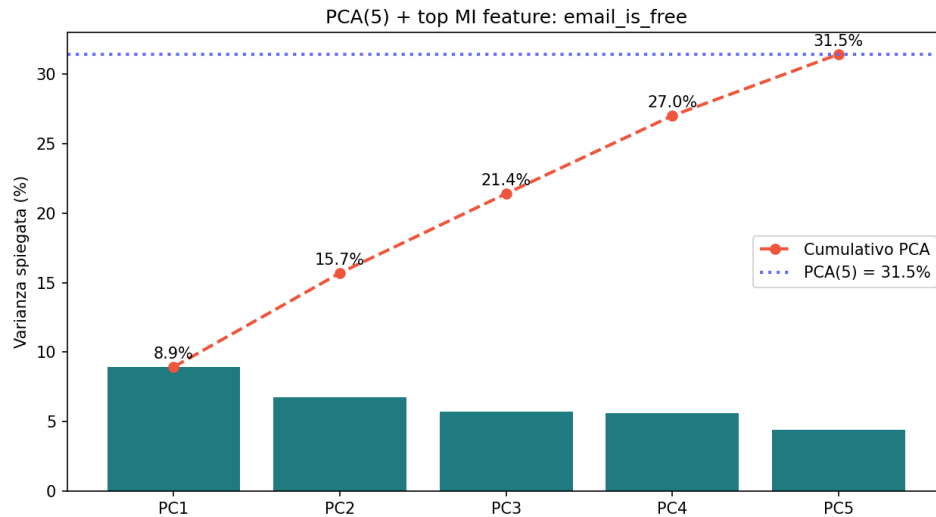
La feature a massima informazione mutua sarà poi preservata esplicitamente come **sesto input** della rappresentazione a 6 dimensioni usata dalla QSVM.

7 Blocco 4 – PCA a 5 componenti + feature supervisionata

L'obiettivo di questo blocco è costruire uno spazio di input **a 6 dimensioni** che combini:

- **5 componenti principali PCA** calcolate su tutte le feature standardizzate;
- **1 feature supervisionata** (la `top_mi_feature`) che ha la massima dipendenza informativa con il target.

La PCA (Principal Component Analysis) è una trasformazione lineare che permette di ridurre la dimensionalità del dataset, preservando la varianza spiegata. La sua funzione è quella di proiettare i dati su un nuovo sistema di assi cartesiani ortogonali, detti “componenti principali”, in modo tale che la varianza dei dati sia massimizzata lungo queste direzioni.



Per la costruzione dello spazio a 6 dimensioni si estraggono i valori della `top_mi_feature` dal DataFrame standardizzato `X_scaled_df` e li si concatena alle 5 componenti PCA.

Questo compromesso aumenta la varianza spiegata rispetto a una PCA a 4 componenti e, allo stesso tempo, mantiene una dimensione supervisionata fortemente predittiva.

8 Blocco 5 – Bilanciamento delle classi

Per ridurre lo sbilanciamento e contenere il costo computazionale del kernel quantistico, lo script lavora su un **sotto-campione bilanciato** del dataset:

1. Si crea un DataFrame `df_6` che contiene le 6 feature finali e il target `fraud_bool`.
2. Si separano i casi legittimi (`fraud_bool = 0`) e fraudolenti (`fraud_bool = 1`).
3. Si campionano **150 esempi per ciascuna classe** (300 in totale), usando `resample` con `replace=False` per la classe maggioritaria e `replace=True` se la classe minoritaria dispone di meno di 150 osservazioni.
4. Si mescolano i 300 esempi ottenuti e si estraggono `X_bal` (feature) e `y_bal` (target).

Questo setup mantiene il problema relativamente piccolo (permette di calcolare il kernel quantistico in tempi ragionevoli) e garantisce classi bilanciate per l'addestramento.

9 Blocco 6 – Scaling angolare per la ZZFeature-Map

La ZZFeatureMap di Qiskit interpreta gli input numerici come **angoli di rotazione** sui qubit. Per questo motivo, le 6 feature in `X_bal` vengono riscalate nell'intervallo $[-\pi, \pi]$ mediante un `MinMaxScaler`:

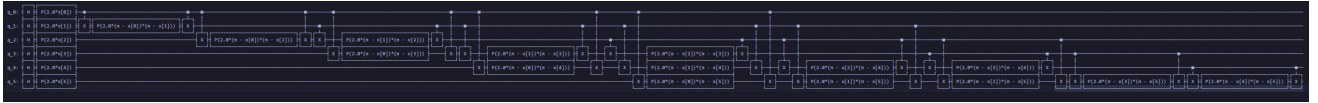
```
1 angle_scaler = MinMaxScaler(feature_range=(-np.pi, np.pi))
2 X_angle = angle_scaler.fit_transform(X_bal)
```

Dopo questa trasformazione, ogni dimensione del vettore è un angolo in radianti, pronto per essere fornito alla ZZFeatureMap a 6 qubit.

10 Blocco 7 – Train/Test split

Si esegue uno split **75/25 stratificato** sul dataset bilanciato:

- `X_train` contiene 225 campioni.
- `X_test` contiene 75 campioni.
- Le classi rimangono bilanciate in entrambi i sottoinsiemi.



11 Blocco 8 – Quantum kernel con Qiskit

Questo è il cuore quantistico della pipeline. Lo script costruisce una ZZFeatureMap a 6 qubit e utilizza il **formalismo compute-uncompute** per stimare una matrice kernel di tipo fidelity.

11.1 ZZFeatureMap a 6 qubit

La feature map è definita come:

```
1 feature_map = ZZFeatureMap(feature_dimension=N_QUBITS, reps=1,
2                             entanglement="full")
```

- `feature_dimension = 6`: lo spazio di input ha 6 dimensioni.
- `reps = 1`: mantiene la profondità del circuito contenuta.
- `entanglement = "full"`: permette interazioni ZZ fra tutti i qubit, aumentando l'espressività del kernel.

11.2 Backend quantistico

Lo script utilizza `AerSimulator` con metodo `statevector` e tenta prima di usare la GPU (`device="GPU"`). Se questa non è disponibile, effettua automaticamente il fallback su CPU.

Il numero di shots per stimare ciascuna entry del kernel è `N_SHOTS = 8192`, un compromesso fra precisione statistica e tempo di esecuzione.

11.3 Compute–uncompute per il kernel

Per due vettori di input $\mathbf{x}_i, \mathbf{x}_j$ (già angolarizzati) si costruisce il circuito compute–uncompute. Lo script definisce due funzioni:

- `compute_kernel_entry(xi, xj, fmap, backend, shots)` – calcola una singola entry.
- `compute_kernel_matrix(X_a, X_b, fmap, backend, shots, symmetric)` – calcola un'intera matrice kernel fra i vettori di `X_a` e `X_b`, sfruttando la simmetria quando `X_a` e `X_b` coincidono.

11.4 Matrici K_{train} e K_{test}

Si calcola innanzitutto la matrice kernel del training set:

```
1 K_train = compute_kernel_matrix(X_train, X_train, feature_map,
2                                backend, symmetric=True)
```

Questa matrice è **simmetrica** per costruzione e viene verificata numericamente. Successivamente si calcola la matrice kernel fra test e train:

```
1 K_test = compute_kernel_matrix(X_test, X_train, feature_map,
2                                backend, symmetric=False)
```

12 Blocco 9 – QSVM e baseline classiche

Una volta ottenute K_{train} e K_{test} , lo script addestra e valuta:

1. **QSVM (6 qubit: PCA5 + MI1)** – un SVC con `kernel="precomputed"` che usa direttamente le matrici kernel quantistiche.
2. **SVM lineare** – un SVC classico con `kernel="linear"` addestrato sui vettori `X_train` (6 dimensioni).
3. **SVM RBF** – un SVC classico con `kernel="rbf"` addestrato sugli stessi vettori.

Per ciascun modello vengono calcolate le principali metriche di classificazione sul test set:

- Accuracy
- Precision
- Recall

- F1-score
- ROC-AUC (tramite probabilità predette)

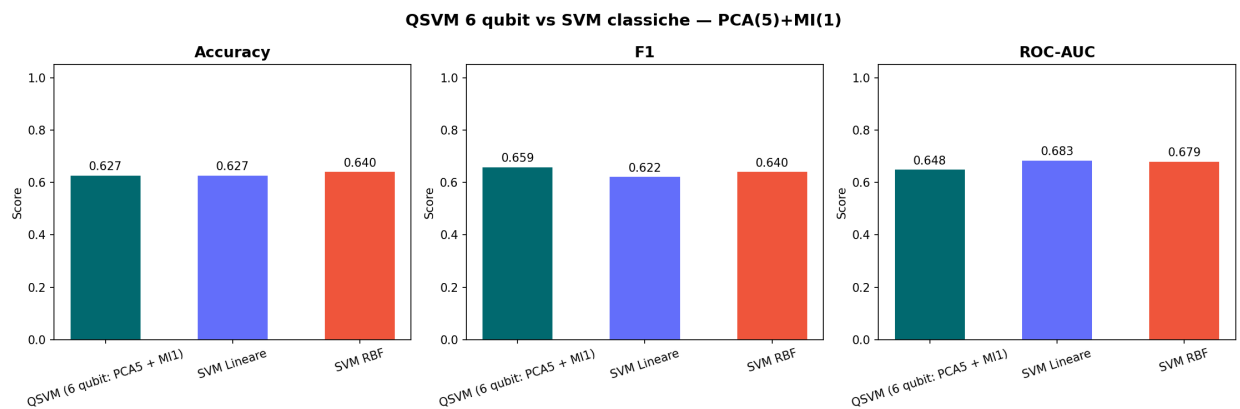
Questo consente un confronto diretto tra le prestazioni della QSVM e delle SVM classiche sullo **stesso spazio di input a 6 dimensioni**.

13 Blocco 10 – Grafici e visualizzazioni

Per una comprensione più intuitiva dei risultati, lo script genera diversi grafici.

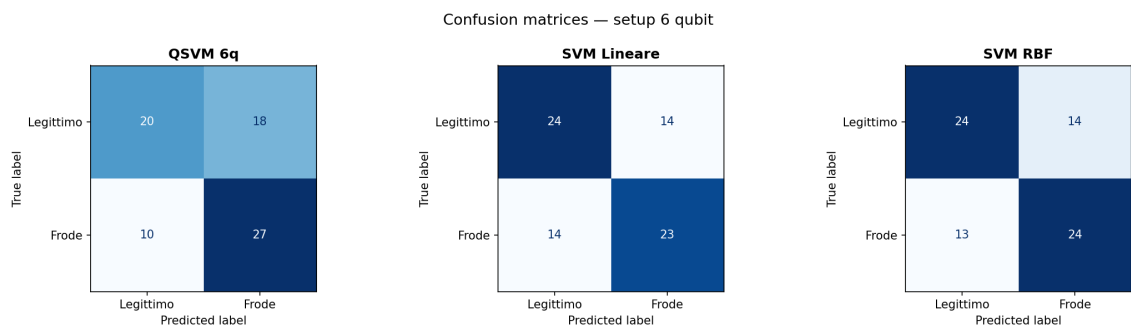
13.1 Confronto delle metriche QSVM vs SVM classiche

Viene generato un grafico a barre per **Accuracy**, **F1** e **ROC-AUC** (benchmark_metrics_6q.png).



13.2 Confusion matrices

Vengono create tre confusion matrices affiancate (confusion_matrices_6q.png), una per ciascun modello.



Questa visualizzazione permette di capire **come** ciascun modello commette errori (es. se tende a produrre molti falsi positivi o falsi negativi).

14 Blocco 11 – Salvataggio risultati e file di output

Al termine dell'esecuzione, lo script salva i file che documentano la parte algoritmica e sperimentale del progetto (CSV, NPY, immagini e un breve `readme_6q.txt`).

15 Considerazioni metodologiche e interpretative

15.1 Motivazioni della scelta PCA(5) + feature MI

Inizialmente è stata testata una configurazione con 4 componenti PCA, che si è rivelata insufficiente per catturare abbastanza informazione (accuracy molto bassa). Si è quindi optato per la combinazione **PCA(5) + 1 feature supervisionata**:

- **PCA(5)** fornisce più varianza spiegata totale;
- **feature top-MI** introduce una dimensione fortemente correlata con il target.

I primi 5 qubit rappresentano una proiezione lineare non supervisionata, mentre il sesto qubit conserva una variabile esplicitamente significativa.

Sono state testate anche combinazioni di 7 qubit (PCA 6 + MI 1) e 8 qubit (PCA 7 + MI 1), ma i risultati non sono migliorati significativamente – in alcuni casi sono persino peggiorati – quindi la configurazione a 6 qubit è stata mantenuta.

15.2 Aspetti critici del kernel quantistico

Anche arricchendo i dati in ingresso, si può incorrere nel fenomeno della **kernel concentration**: il modello quantistico fatica a cogliere le vere differenze tra i dati, rendendo difficile la classificazione. Le impostazioni `reps=1` e `entanglement="full"` sono state scelte proprio per trovare un buon compromesso: un modello abbastanza complesso da apprendere, ma non così tanto da “bloccarsi” a causa di questo problema.

15.3 Interpretazione del confronto con le SVM classiche

La QSVM viene confrontata con:

- una **SVM lineare** (margine lineare nello spazio a 6 dimensioni);
- una **SVM RBF** (kernel non lineare classico).

In molti scenari il modello SVM RBF ottiene prestazioni competitive o superiori rispetto alla QSVM, a fronte di un costo computazionale significativamente inferiore. Questo risultato mette in luce che il **vantaggio quantistico non è garantito a priori** e deve essere verificato sperimentalmente caso per caso.